DWH MCP EXPLORER

TESTING & VALIDATION REPORT

Stage 6.9.3 | DWH MCP Explorer | English edition

Evidence baseline: 7 November 2026 • Validation scope: Stages 5–6

A traceable summary of the MCP server validation completed during development. It distinguishes direct observations, mock-based checks and checks performed through Claude Desktop; it is not an external audit or a production certification.

1. Executive summary and release assessment

The tested baseline exposes 21 MCP tools: 13 PostgreSQL / SQL / DWH tools and 8 Azure Databricks tools. Stage 6.6 established on-demand Knowledge Object composition, while Stage 6.7 added exact single-column grain validation via a read-only Databricks SQL statement.

Stage 6.8 was completed after updating legacy registry assertions to accommodate 21 tools. The observed smoke, regression and negative scenarios passed. This finding is limited to the evidence and environments described below.

Validation dimension	Observed result	Evidence class
Stage 6.8.1 — recent regression	7 / 7 selected tests passed	Local automated
Stage 6.8.2 — mixed tool registry	21 names; required cross-platform tools present	In-process MCP
Stage 6.8.3 — negative paths	3 / 3 expected failures detected	Mock-based
Stage 6.8.4 — error propagation	MCP CallToolResult.is_error = True	Mock + MCP client
Stage 6.8.5 — legacy regression	5 / 5 selected Stage 5 tests passed	Local automated
Databricks Knowledge Object	Metadata + 50 table and 50 column lineage events	Live + Claude Desktop
Databricks grain check	10,000 rows, 10,000 distinct order_id, zero NULL and duplicate excess	Live + Claude Desktop

2. Test strategy, boundaries and environments

Unit and mock tests isolate SQL generation, provider response parsing, tool registration, data composition and error handling; they deliberately do not establish availability of remote services.

Live Databricks tests use the previously configured Azure Databricks workspace, the MARKETPLACE_LAB OAuth profile, Unity Catalog (marketplace_lab) and SQL Warehouse wh-marketplace-lab. These tests were read-only but may consume serverless compute. The actual account credentials and private endpoint details are deliberately omitted.

Claude Desktop E2E tests exercised the local DWH Metadata Explorer MCP connector, confirming that Claude could discover and invoke composite tools and interpret the returned structured results. The connector initially was not attached to a conversation; enabling it and changing tool access settings resolved discovery.

Earlier PostgreSQL and SQL repository tests provide legacy coverage. The Stage 6.8.2 mixed-platform test checks that tool names coexist in one server, not that both databases are concurrently reachable.

3. Stage 6.8 — detailed regression evidence

3.1 Recent regression suite: 7/7

Suite / test	Outcome	What was verified
test_stage6_grain_sql	PASS	Quoted identifiers, valid SELECT, rejected unsafe table input
test_stage6_grain_provider	PASS	SDK response parsing and grain result with mocks
test_stage6_grain_registry	PASS	databricks_validate_grain registered; 21-tool baseline
test_stage6_grain_tool	PASS	MCP call returns expected structured content
test_stage6_knowledge	PASS	Mock providers combine metadata and both lineage sets
test_stage6_knowledge_registry	PASS	Knowledge tool and grain tool present in registry
test_stage6_knowledge_tool	PASS	Composite tool content and JSON response via MCP

3.2 Legacy Stage 5 suite: 5/5

Suite / test	Outcome	What was verified
test_stage5_shared_metadata	PASS	Metadata reused; target queried once; reporting contract preserved
test_stage5_shared_lineage	PASS	Lineage shared with reconciliation without repeated parsing
test_stage5_composition	PASS	Composite tool invocation and response contract intact
test_stage5_error_classification	PASS	Five expected error categories classified
test_stage5_reporting_reuse	PASS	Lineage reused and report output unchanged

During regression testing, test_stage6_knowledge_registry initially required exactly 20 tools, and test_stage5_composition required exactly 17. Both assertions were updated to tolerate additive tools and explicitly require the critical names. Subsequent reruns passed. An earlier psycopg import sequence was interrupted by KeyboardInterrupt; a standalone rerun of the registry test passed, so no persistent psycopg defect was demonstrated.

The Stage 5 composition test still printed the historical message “PASS: 17 MCP tools registered” even though the live registry contained 21 names; its message is a stale label, not proof of only 17 tools.

4. Negative tests and MCP error propagation

Scenario	Expected system behavior	Observed
Unsafe table identifier	Reject before calling SQL Warehouse	ValueError / PASS
Databricks statement FAILED	Surface failed SQL execution	RuntimeError / PASS
Successful statement with no rows	Reject missing result	RuntimeError / PASS
Simulated provider RuntimeError	Return an MCP error, never false success	is_error=True / PASS

In the simulated tool failure, the server logged RuntimeError: Simulated Databricks SQL execution failure; the MCP layer exposed UnexpectedToolError: Error executing tool databricks_validate_grain, and the client observed

is_error=True. The log stack trace is expected diagnostic output for the injected failure, not a failed test.

5. Live integration and Claude Desktop acceptance

Acceptance check	Observed evidence
Unity Catalog inspection	gold.fact_orders described as MANAGED / DELTA with 26 columns; reported grain one row per order_id.
Table/column lineage	Both system.access.* queries returned status completed and 50 events for gold.fact_orders; 45-day lookback and LIMIT 50 in current SQL.
Composite Knowledge Object	Direct provider integration passed; Claude Desktop invoked databricks_investigate_object and produced a technical lineage report.
Read-only grain SQL	Exact COUNT / COUNT DISTINCT / NULL count returned 10,000 / 10,000 / 0; inferred duplicate excess = 0.
Grain MCP E2E	Claude Desktop called databricks_validate_grain and reported PASSED for order_id with no data changes.

The grain finding supersedes a potential anomaly suggested by approximate Unity Catalog distinct-count statistics. The exact aggregate query found no duplicate or NULL order_id values in the tested snapshot. This is not a guarantee against future duplicates or violations of other constraints.

The Knowledge Object v1 returns an empty execution_context; job, notebook and SQL identifiers observed by Claude were extracted from lineage event records rather than resolved to names or code. Claude also separated confirmed source→target dependency edges from read-only events without a recorded target.

6. Acceptance criteria and known limitations

• Acceptance: 21 tool names are discoverable in one MCP registry, including key PostgreSQL/DWH and Databricks interfaces.

• Acceptance: new Knowledge and Grain tools can be invoked using an in-process MCP client, returning the expected contracts.

• Acceptance: read-only live Databricks checks and Claude Desktop scenarios succeeded for marketplace_lab.gold.fact_orders.

• Acceptance: selected legacy and new regression suites passed after updating stale registry-count assumptions.

• Limitation: testing did not run a full, exhaustive test discovery over every repository test, nor a load/stress test or external security audit.

• Limitation: remote-service auth, SQL Warehouse availability, local PostgreSQL runtime and Claude plan limits can vary between sessions.

• Limitation: lineage coverage is capture-based, capped at 50 events per call with a 45-day lookback; truncated=false at a query LIMIT does not establish completeness.

• Limitation: grain validation handles one simple business-key column and does not validate composite keys or general SQL mapping consistency.

• Limitation: cross-platform registry validation does not prove simultaneous connectivity to PostgreSQL and Databricks.

7. Traceability and next steps

Sources: user-shared terminal output for Stage 6.6–6.8 tests, screenshots of Databricks SQL Editor and Claude Desktop technical reports, and the Stage 6.9.1–6.9.2 architecture/implementation baselines. Counts reflect those concrete observations; no invented execution run IDs or coverage percentages are included.

Stage 6.9.4 should capture setup and reproducible validation commands without publishing any .env contents, OAuth/token caches, private endpoints or local credentials. Stage 6.9.5 should summarize the project for a technical portfolio. Before GitHub publication, rerun relevant tests against the final checked-in code, review dependency locks and scan for secrets.

Source examples / command inventory

```
uv run python -m tests.test_stage6_grain_sql
uv run python -m tests.test_stage6_grain_provider
uv run python -m tests.test_stage6_knowledge_tool
uv run python -m tests.test_stage6_cross_platform_registry
uv run python -m tests.test_stage6_grain_negative
uv run python -m tests.test_stage6_mcp_error_response
uv run python -m tests.test_stage5_composition
```

ASSESSMENT: The selected Stage 6 implementation and regression checks are successful. Remaining risks and untested dimensions are documented; this is a validated development baseline, not a production-readiness certification.